

Chapter 25: Java Streams API

Personal Notes

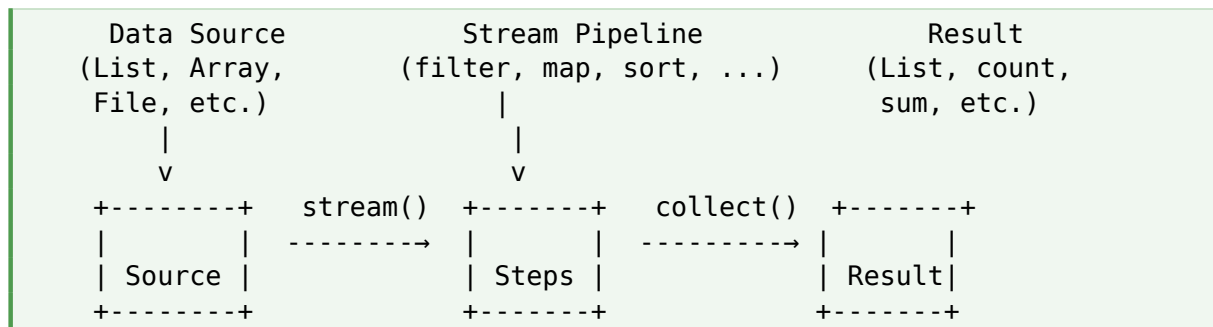
The Streams API, introduced in Java 8, is one of the most powerful additions to the language. It lets you process collections of data with clean, expressive, functional-style pipelines — filtering, transforming, grouping, and summarizing in one fluent chain.

Combined with the lambdas you learned in Chapter 24, Streams turn what used to be 20-line loops into single readable lines. They're a core skill for modern Java — used everywhere from backend code to Spring to Android.

1. What is a Stream?

A Stream is a SEQUENCE of elements that supports VARIOUS OPERATIONS performed in a PIPELINE. Streams are NOT data structures — they don't store data. They flow data THROUGH a series of operations.

The Mental Model



Streams don't modify the original data. They produce a NEW result based on the operations you ask for.

2. Why Streams? (Before vs After)

Streams shine for data-processing tasks. Look at a simple example:

Task: Get the names of students with marks ≥ 80 , sorted alphabetically.

Before — Imperative Style

```

List<String> result = new ArrayList<>();
for (Student s : students) {
    if (s.marks >= 80) {
        result.add(s.name);
    }
}
  
```

```
Collections.sort(result);  
return result;
```

After — Stream Style

```
return students.stream()  
    .filter(s -> s.marks >= 80)  
    .map(s -> s.name)  
    .sorted()  
    .collect(Collectors.toList());
```

Both do the same thing — but the stream version reads almost like English. "Take the students, keep the ones with marks ≥ 80 , get their names, sort them, collect into a list."

What changes with streams: (1) You describe WHAT you want, not HOW to do it. (2) Less boilerplate — no temp lists, no explicit loops. (3) Operations chain together naturally. (4) Code reads top-to-bottom like a pipeline.

3. The Stream Pipeline

Every stream pipeline has THREE parts:

Source (creates)	Intermediate Ops (transforms)	Terminal Op (consumes)
 v	 v	 v
.stream() Stream.of(...)	.filter, .map, .sorted, .distinct, .limit, .skip, .peek, .flatMap	.collect, .count, .reduce, .forEach, .findFirst, etc.

- Source — where the data comes from (a collection, array, file, etc.)
- Intermediate Operations — return a new Stream; you can chain as many as you want
- Terminal Operation — produces a final result and ENDS the stream

Important rule: A stream can be used ONLY ONCE. Once you call a terminal operation, the stream is consumed and can't be reused. Just create a new stream from the source if you need to run another pipeline.

3.1 Lazy Evaluation

Intermediate operations are LAZY — they don't actually run until a terminal operation triggers them. Without a terminal op, nothing happens at all.

```
// This does NOTHING — no terminal op  
students.stream()  
    .filter(s -> { System.out.println("filtering"); return s.marks >  
80; });
```

```
// Now it runs – collect() is the terminal op
List<Student> result = students.stream()
    .filter(s -> { System.out.println("filtering"); return s.marks >
80; })
    .collect(Collectors.toList());
```

4. Creating Streams

There are many ways to create a stream. Most common ones:

4.1 From a Collection

```
List<String> names = Arrays.asList("Aman", "Riya", "Karan");
Stream<String> s = names.stream();
```

4.2 From an Array

```
String[] arr = {"a", "b", "c"};
Stream<String> s = Arrays.stream(arr);

int[] nums = {1, 2, 3, 4, 5};
IntStream is = Arrays.stream(nums);
```

4.3 From Specific Values

```
Stream<String> s = Stream.of("apple", "banana", "cherry");
Stream<Integer> nums = Stream.of(1, 2, 3, 4, 5);
```

4.4 Empty Stream

```
Stream<String> empty = Stream.empty();
```

4.5 Infinite Streams (use with limit)

```
// Stream.iterate – produces 0, 2, 4, 6, 8, ...
Stream.iterate(0, n -> n + 2).limit(5).forEach(System.out::println);

// Stream.generate – produces 5 random numbers
Stream.generate(Math::random).limit(5).forEach(System.out::println);
```

4.6 Range Streams

```
IntStream.range(1, 6).forEach(System.out::println);           // 1 2 3 4 5
IntStream.rangeClosed(1, 5).forEach(System.out::println);     // 1 2 3 4 5
IntStream.rangeClosed(1, 100).sum();                           // 5050
```

5. Intermediate Operations

These are the building blocks of your pipeline. Each returns a new Stream.

5.1 filter — Keep What Matches

Pass a Predicate. Only elements where it returns true survive.

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6, 7);

nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::print);    // 2 4 6
```

5.2 map — Transform Each Element

Pass a Function. Each element is replaced by the function's output.

```
List<String> names = Arrays.asList("aman", "riya", "karan");

names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);    // AMAN  RIYA  KARAN

// Map to a different type:
List<Integer> lengths = names.stream()
    .map(String::length)
    .collect(Collectors.toList());    // [4, 4,
5]
```

5.3 sorted — Sort the Stream

```
List<Integer> nums = Arrays.asList(5, 2, 8, 1, 9);

nums.stream().sorted().forEach(System.out::print);
// 1 2 5 8 9

// With a custom Comparator:
nums.stream()
    .sorted(Comparator.reverseOrder())
    .forEach(System.out::print);
// 9 8 5 2 1
```

5.4 distinct — Remove Duplicates

```
Stream.of(1, 2, 2, 3, 3, 3, 4)
    .distinct()
    .forEach(System.out::print);    // 1 2 3 4
```

5.5 limit and skip — Take or Drop Elements

```
Stream.of(1, 2, 3, 4, 5, 6, 7, 8)
    .limit(3)                // take only first 3
    .forEach(System.out::print); // 1 2 3

Stream.of(1, 2, 3, 4, 5, 6, 7, 8)
    .skip(3)                // drop first 3
    .forEach(System.out::print); // 4 5 6 7 8
```

5.6 peek — Debugging Helper

Lets you see what's flowing through the pipeline without modifying it. Useful when streams aren't producing what you expect.

```
nums.stream()
    .filter(n -> n > 2)
    .peek(n -> System.out.println("After filter: " + n))
    .map(n -> n * 10)
    .peek(n -> System.out.println("After map: " + n))
    .collect(Collectors.toList());
```

5.7 Quick Reference Table

Operation	Takes	What it does
filter	Predicate<T>	Keep elements matching the test
map	Function<T, R>	Transform each element
sorted	Optional Comparator	Sort the elements
distinct	—	Remove duplicates
limit	long n	Keep first n elements
skip	long n	Drop first n elements
peek	Consumer<T>	Apply action without changing flow (debug)
flatMap	Function<T, Stream>	Flatten nested streams

6. Terminal Operations

Terminal operations CONSUME the stream and produce a final result. Once called, the stream is done.

6.1 forEach — Do Something with Each Element

```
names.stream().forEach(System.out::println);
nums.stream().forEach(n -> System.out.println("Got: " + n));
```

6.2 collect — Gather Into a Collection

The most-used terminal op. Uses a Collector to bundle results into a List, Set, Map, etc.

```
// Collect into a List
List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());

// Collect into a Set
Set<Integer> uniqueEvens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toSet());

// Java 16+ shortcut
List<Integer> evens2 = nums.stream()
    .filter(n -> n % 2 == 0)
    .toList();    // shorter
```

6.3 count — How Many?

```
long count = nums.stream()
    .filter(n -> n > 5)
    .count();
```

6.4 reduce — Combine Into One Value

Repeatedly combine elements using a binary operation. The classic example: sum, product, max.

```
// Sum using reduce
int sum = Stream.of(1, 2, 3, 4, 5)
    .reduce(0, Integer::sum);    // 15
    //      ^      ^
    //  initial  combiner

// Max using reduce
Optional<Integer> max = Stream.of(5, 2, 8, 1, 9)
    .reduce(Integer::max);    // Optional[9]

// Product
int product = Stream.of(1, 2, 3, 4)
    .reduce(1, (a, b) -> a * b);    // 24
```

6.5 min and max

```
Optional<Integer> max = nums.stream().max(Comparator.naturalOrder());
Optional<Integer> min = nums.stream().min(Comparator.naturalOrder());

max.ifPresent(System.out::println);
```

6.6 anyMatch, allMatch, noneMatch — Boolean Tests

```
boolean hasNegative = nums.stream().anyMatch(n -> n < 0);    // is ANY
negative?
boolean allPositive = nums.stream().allMatch(n -> n > 0);    // are ALL
positive?
boolean noZero = nums.stream().noneMatch(n -> n == 0);      // are NO
zeros?
```

6.7 findFirst and findAny

```
Optional<Integer> first = nums.stream()
    .filter(n -> n > 5)
    .findFirst();    // first match

Optional<Integer> any = nums.parallelStream()
    .filter(n -> n > 5)
    .findAny();      // any match (faster for
parallel)
```

7. Collectors — The Power of collect()

The Collectors utility class has dozens of pre-built ways to gather stream results. These are some of the most-used ones in real code.

7.1 toList, toSet, toMap

```
// To List
List<String> list = names.stream().collect(Collectors.toList());

// To Set
Set<String> set = names.stream().collect(Collectors.toSet());

// To Map (key → value)
Map<String, Integer> nameToLength =
    names.stream()
        .collect(Collectors.toMap(name -> name, String::length));
```

7.2 joining — Combine Into a String

```
String result = Stream.of("Java", "is", "fun")
```

```

        .collect(Collectors.joining(" "));
// "Java is fun"

String csv = Stream.of("Aman", "Riya", "Karan")
    .collect(Collectors.joining(", ", "[", "]"));
// "[Aman, Riya, Karan]"

```

7.3 counting

```

Long count = students.stream()
    .filter(s -> s.marks > 80)
    .collect(Collectors.counting());
// Equivalent to .count(), but composable with grouping

```

7.4 groupingBy — The Most Powerful One

Group elements by a key, just like SQL's GROUP BY.

```

// Group students by grade
Map<String, List<Student>> byGrade =
    students.stream()
        .collect(Collectors.groupingBy(s -> s.grade));

// Count students per grade
Map<String, Long> countPerGrade =
    students.stream()
        .collect(Collectors.groupingBy(
            s -> s.grade,
            Collectors.counting()
        ));

// Average marks per grade
Map<String, Double> avgPerGrade =
    students.stream()
        .collect(Collectors.groupingBy(
            s -> s.grade,
            Collectors.averagingInt(s -> s.marks)
        ));

```

7.5 partitioningBy — Split Into Two Groups

Like groupingBy but always produces exactly 2 groups based on a Predicate (true/false).

```

Map<Boolean, List<Student>> split =
    students.stream()
        .collect(Collectors.partitioningBy(s -> s.marks >= 80));

List<Student> passed = split.get(true);
List<Student> failed = split.get(false);

```


7.6 summingInt / averagingInt

```
int totalMarks = students.stream()
    .collect(Collectors.summingInt(s -> s.marks));

double avgMarks = students.stream()
    .collect(Collectors.averagingInt(s ->
s.marks));
```

8. Numeric Streams

Working with primitives (int, long, double) through `Stream<Integer>` involves a lot of boxing/unboxing — slow and ugly. Java provides `IntStream`, `LongStream`, and `DoubleStream` for efficient numeric work.

```
// IntStream – built for primitive ints
IntStream.range(1, 11).sum();           // 55
IntStream.rangeClosed(1, 100).sum();    // 5050

int max = IntStream.of(5, 2, 8, 1, 9).max().getAsInt();
double avg = IntStream.of(1, 2, 3, 4, 5).average().getAsDouble();

// Convert Stream<Integer> to IntStream
int total = Stream.of(1, 2, 3, 4, 5)
    .mapToInt(Integer::intValue)
    .sum();
```

When to reach for `IntStream`: Whenever you're doing arithmetic on a stream of numbers — sum, average, max, min on big lists. `Stream<Integer>` wraps each element in an `Integer` object, which is wasteful. `IntStream` uses raw ints — much faster.

9. Parallel Streams

By calling `.parallelStream()` instead of `.stream()` (or `.parallel()` on an existing stream), Java will split the work across multiple CPU cores automatically. For big data sets, this can be a major speedup.

```
// Sequential
long count = bigList.stream()
    .filter(s -> isExpensive(s))
    .count();

// Parallel – uses multiple cores
long count = bigList.parallelStream()
    .filter(s -> isExpensive(s))
    .count();
```

When NOT to use parallel streams: Small data sets (overhead dominates). Operations with side effects (race conditions). Order-sensitive operations like `findFirst`. I/O-heavy work (use

CompletableFuture instead). The rule: parallelize ONLY when you have CPU-bound work on a big dataset and you've measured it actually helps.

10. Real-World Example

Here's a complete example that ties everything together.

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    String department;
    double salary;

    Employee(String n, String d, double s) {
        this.name = n; this.department = d; this.salary = s;
    }
}

public class Main {
    public static void main(String[] args) {
        List<Employee> emps = Arrays.asList(
            new Employee("Aman", "Engineering", 95000),
            new Employee("Riya", "Engineering", 88000),
            new Employee("Karan", "Sales", 72000),
            new Employee("Priya", "Sales", 81000),
            new Employee("Dev", "HR", 60000)
        );

        // 1. Total salary
        double total = emps.stream()
            .mapToDouble(e -> e.salary)
            .sum();

        // 2. Top earner
        Employee top = emps.stream()
            .max(Comparator.comparing(e -> e.salary))
            .orElseThrow();

        // 3. Names of employees earning > 80k, sorted
        List<String> highEarners = emps.stream()
            .filter(e -> e.salary > 80000)
            .map(e -> e.name)
            .sorted()
            .collect(Collectors.toList());

        // 4. Average salary per department
```

```

        Map<String, Double> avgPerDept = emps.stream()
            .collect(Collectors.groupingBy(
                e -> e.department,
                Collectors.averagingDouble(e -> e.salary)
            ));

        // 5. Comma-separated list of all names
        String allNames = emps.stream()
            .map(e -> e.name)
            .collect(Collectors.joining(", "));
    }
}

```

11. Common Mistakes

Mistake 1: Reusing a stream

```

Stream<Integer> s = nums.stream();
long count = s.count();           // ok
long count2 = s.filter(...).count(); // ERROR – stream already consumed

```

Create a new stream each time, or extract the source first.

Mistake 2: Forgetting the terminal operation

```

// This does NOTHING – no terminal op
nums.stream().filter(n -> n > 5).map(n -> n * 2);

// Fix: add a terminal op
nums.stream().filter(n -> n > 5).map(n -> n *
2).forEach(System.out::println);

```

Mistake 3: Side effects inside filter/map

```

// BAD – modifying outer state from inside a stream
List<Integer> kept = new ArrayList<>();
nums.stream()
    .filter(n -> { kept.add(n); return n > 0; })
    .count();

// GOOD – let the stream collect for you
List<Integer> result = nums.stream()
    .filter(n -> n > 0)
    .collect(Collectors.toList());

```

Mistake 4: Using stream where a loop is clearer

Not every loop should become a stream. If the logic involves complex conditions, multiple variables, or early exits, a regular loop is often more readable. Use streams when they make things CLEARER, not just shorter.

Mistake 5: Wrong choice between map and flatMap

```
// Use map when each element produces ONE result
.map(s -> s.toUpperCase())

// Use flatMap when each element produces a STREAM (and you want them flattened)
.flatMap(s -> Arrays.stream(s.split(" ")))
```

Mistake 6: Premature parallelization

Don't use `parallelStream()` everywhere. For small streams, it's actually slower because of thread setup overhead. Always measure before parallelizing.

12. Best Practices

- Use streams for data PROCESSING — filtering, mapping, summarizing. Not for everything.
- Keep pipelines short — break long ones into intermediate methods
- Use method references (`::`) wherever you can for clarity
- Use `IntStream/LongStream/DoubleStream` for numeric operations to avoid boxing
- Prefer `toList()`, `toSet()` shortcuts in Java 16+ over `Collectors.toList()`
- Don't mutate outer state inside lambdas — keep streams pure
- Use `peek()` only for debugging, never for production logic
- Only parallelize after measuring — and only for big, CPU-bound work

13. Quick Reference

Operation	What it does
stream()	Create a stream from a collection
Stream.of(...)	Create a stream from specific values
filter(predicate)	Keep elements matching the test
map(function)	Transform each element
flatMap(function)	Transform each element into a stream, then flatten
sorted() / sorted(c)	Sort naturally, or by Comparator
distinct()	Remove duplicates
limit(n) / skip(n)	Take first n / drop first n
peek(consumer)	Inspect elements without changing them (debug)

Operation	What it does
forEach(consumer)	Apply an action to each element
collect(collector)	Gather results into a List, Set, Map, or string
count()	How many elements
reduce(op)	Combine all elements into one value
min/max(comparator)	Smallest or largest element
anyMatch / allMatch / noneMatch	Boolean checks
findFirst / findAny	Get an element (Optional)
Collectors.toList()	Collect to a List
Collectors.toMap(k, v)	Collect to a Map
Collectors.groupingBy(k)	Group by key — like SQL GROUP BY
Collectors.joining(sep)	Join elements into a String

14. Practice Problems

Try these in order. Each one combines a few stream operations.

1. Given a `List<Integer>`, return a new list with only even numbers, doubled.
2. Given a `List<String>`, count how many strings start with the letter "A".
3. Find the sum of squares of all numbers from 1 to 100 using `IntStream`.
4. Given a `List<String>`, return a comma-separated string of all names in uppercase.
5. Find the max element in a list, throwing an exception if empty.
6. Given a `List<Student>`, group students by their grade and return `Map<Grade, List<Student>>`.
7. Count how many students passed (marks ≥ 40) using `partitioningBy`.
8. Given a `List<String>`, return distinct words sorted by length, then alphabetically.
9. Given an `int[]` `nums`, find the average without converting to `Integer`.
10. Given a `List<Employee>`, find the employee with the highest salary in each department.
11. Build a frequency map: given a `List<String>`, return `Map<String, Long>` with counts.
12. Given a `List<List<Integer>>`, flatten and return a `List<Integer>` of all numbers.
13. Generate the first 10 Fibonacci numbers using `Stream.iterate()`.

14. Check if all employees in a list are earning over 50000 using `allMatch`.
15. Take a sentence, split into words, count how many are longer than 5 chars.

15. Final Takeaway

The Streams API changes how you think about Java code. Instead of writing loops that DO things, you describe a pipeline of what you WANT. The result is shorter, clearer, and often faster.

Master streams and your code starts to look like adult Java — the kind you see in Spring, in Android, in real backend systems. The investment in learning streams pays back every single day after that.

Key Takeaway: A stream pipeline = source → intermediate ops (filter, map, sorted) → terminal op (collect, count, reduce). Master these 4 operations: filter, map, sorted, collect — they handle 70% of real-world stream code. Add `groupBy` and you can replace half your for-loops with declarative pipelines.

16. What's Next?

Now that streams are clear, the next Java topics worth exploring are:

- `Optional<T>` — the safer alternative to null, used heavily with streams
- Collectors in depth — `toMap` with merge functions, downstream collectors
- File I/O with streams — `Files.lines()` for line-by-line processing
- `CompletableFuture` — async programming for non-blocking work
- Multithreading — `Thread`, `Runnable`, `synchronized`, `ExecutorService`
- Java 9+ features — `var`, modules, text blocks, switch expressions